

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И.
УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

ОТЧЕТ

по лабораторной работе №1

по дисциплине “Алгоритмы и структуры данных”

Тема: Сравнительный анализ алгоритмов сжатия данных без
потерь: RLE, BWT, MTF, LZ77/LZW и кодирование Хаффмана

Студент гр. 4354

_____ Шупряков В.В.

Преподаватель

_____ Пестерев Д.О.

Санкт-Петербург

2026

Цель работы

Реализация и сравнительный анализ алгоритмов сжатия данных без потерь, включая RLE, преобразование Барроуза-Уиллера (BWT), Move-to-Front (MTF), методы Лемпеля-Зива (LZ77, LZ78, LZW, LZSS) и энтропийное кодирование Хаффмана, а также исследование эффективности их комбинаций на различных типах данных.

1 Теоретическая часть

1.1 Алгоритм Run-Length Encoding (RLE)

Алгоритм Run-Length Encoding (RLE) представляет собой метод сжатия данных без потерь, основанный на замене последовательностей одинаковых символов парой «длина последовательности – символ». Например, последовательность

AAAAA

может быть представлена в виде

(5, A).

В реализованной версии алгоритма используются управляющий символ длиной M_s байт и код символа длиной M_s байт. Для поддержки неповторяющихся последовательностей применяется старший бит управляющего символа. Если старший бит равен нулю, далее записывается повторяющаяся последовательность, если единице — последовательность различных символов.

Алгоритм показывает высокую эффективность при обработке данных, содержащих длинные серии одинаковых символов. Наиболее хорошо RLE подходит для черно-белых изображений, изображений в оттенках серого и неко-

торых типов текстовых данных. Для случайных данных и уже сжатых файлов алгоритм может приводить к увеличению размера сообщения.

Временная сложность алгоритма составляет

$$O(n),$$

а пространственная сложность также оценивается как

$$O(n),$$

где n — размер входной последовательности.

При применении RLE непосредственно к UTF-8 тексту возникает проблема, связанная с тем, что один символ может кодироваться переменным количеством байт. В результате одинаковые символы могут не распознаваться как одинаковые последовательности байт. Для решения данной проблемы текст предварительно переводился в кодировку UTF-32, в которой каждый символ имеет фиксированную длину 4 байта.

1.2 Энтропия сообщения

Энтропия сообщения характеризует среднее количество информации, приходящееся на один символ сообщения. Для оценки энтропии используется формула Шеннона:

$$H(X) = - \sum_{i=1}^n p_i \log_2 p_i,$$

где p_i обозначает вероятность появления символа i , а n — количество различных символов алфавита.

Чем меньше значение энтропии, тем выше потенциальная степень сжатия сообщения. При изменении длины кодируемого символа изменяется распределение вероятностей, вследствие чего значение энтропии также может изменяться.

1.3 Преобразование Move-To-Front (MTF)

Преобразование Move-To-Front является обратимым преобразованием, заменяющим символ его индексом в динамически изменяемом списке символов. В начале работы формируется список всех возможных символов алфавита. Для каждого символа входной последовательности определяется его позиция в текущем списке, после чего индекс записывается в выходной поток, а сам символ переносится в начало списка.

После применения преобразования Барроуза–Уиллера одинаковые символы оказываются сгруппированными, благодаря чему MTF генерирует большое количество малых значений индексов. Это уменьшает энтропию сообщения и повышает эффективность последующего энтропийного кодирования.

Временная сложность преобразования оценивается как

$$O(n \cdot t),$$

где n — длина сообщения, а t — размер алфавита.

1.4 Кодирование Хаффмана

Алгоритм Хаффмана представляет собой метод энтропийного кодирования без потерь, основанный на использовании префиксных кодов переменной длины. Более часто встречающимся символам назначаются короткие коды, а редким — длинные.

Построение дерева Хаффмана выполняется на основе частот появления символов. На каждом шаге выбираются два узла с минимальными весами, которые объединяются в новый узел. Процесс продолжается до тех пор, пока не будет построено полное дерево.

Средняя длина кодового слова определяется выражением

$$L = \sum_{i=1}^n p_i l_i,$$

где p_i — вероятность появления символа, а l_i — длина соответствующего кодового слова.

Для корректного декодирования необходимо сохранить либо само дерево Хаффмана, либо таблицу кодов, либо длины канонических кодов. Построение дерева выполняется за время

$$O(n \log n).$$

1.5 Арифметическое кодирование

Арифметическое кодирование представляет сообщение в виде одного вещественного числа из интервала

$$[0, 1).$$

На каждом шаге текущий интервал сужается пропорционально вероятностям символов. Пусть текущий интервал имеет вид

$$[l, r).$$

Тогда после обработки очередного символа новые границы вычисляются по формулам:

$$l' = l + (r - l) \cdot P_{low},$$

$$r' = l + (r - l) \cdot P_{high},$$

где P_{low} и P_{high} обозначают накопленные вероятности символа.

Основной недостаток алгоритма связан с ограниченной точностью представления вещественных чисел, особенно при обработке длинных сообщений.

1.6 Преобразование Барроуза–Уиллера (BWT)

Преобразование Барроуза–Уиллера является обратимым преобразованием, предназначенным для повышения эффективности последующего сжатия данных. На первом этапе строятся все циклические сдвиги строки, затем полученные строки сортируются лексикографически, после чего формируется последний столбец полученной матрицы.

После применения BWT одинаковые символы оказываются расположенными рядом друг с другом, что значительно повышает эффективность алгоритмов RLE, MTF и энтропийного кодирования.

Прямое построение матрицы Барроуза–Уиллера требует

$$O(n^2)$$

времени и памяти. Использование суффиксного массива позволяет уменьшить сложность до

$$O(n \log n).$$

1.7 Алгоритм LZ77

Алгоритм LZ77 использует механизм скользящего окна и заменяет повторяющиеся подстроки ссылками вида

$$(offset, length, nextSymbol),$$

где *offset* обозначает насколько символы находятся назад, *length* — длину совпадения, а *nextSymbol* — следующий символ после совпадения.

Эффективность алгоритма возрастает при наличии большого количества повторяющихся фрагментов данных.

При наивной реализации временная сложность алгоритма может достигать

$$O(n \cdot w),$$

где w — размер окна поиска.

Для корректного декодирования алгоритмов семейства LZ необходимо сохранять параметры кодирования, такие как размер буфера поиска, размер окна просмотра и используемый формат представления ссылок и литералов.

1.8 Алгоритм LZSS

Алгоритм LZSS является модификацией LZ77. Основное отличие заключается в том, что короткие совпадения не заменяются ссылками, а записываются как обычные символы. Это позволяет уменьшить избыточность выходного потока.

Для различения литералов и ссылок используется специальный управляющий флаг.

1.9 Алгоритм LZ78

Алгоритм LZ78 строит словарь фраз непосредственно в процессе кодирования. Каждая новая запись кодируется в виде пары

$$(index, symbol),$$

где *index* представляет индекс существующей записи словаря, а *symbol* — новый символ.

По мере обработки сообщения словарь постепенно расширяется, что позволяет эффективно кодировать повторяющиеся последовательности.

1.10 Алгоритм LZW

Алгоритм LZW является развитием метода LZ78. В отличие от LZ78, в выходной поток записываются только индексы словаря. Начальный словарь содержит все одиночные символы алфавита.

Преимуществом алгоритма является отсутствие необходимости сохранять словарь в сжатом сообщении, поскольку он одинаково восстанавливается как при кодировании, так и при декодировании.

Средняя временная сложность алгоритма оценивается как

$$O(n),$$

при использовании эффективной структуры хранения словаря.

1.11 Канонические коды Хаффмана

Канонические коды Хаффмана позволяют существенно уменьшить объем метаданных по сравнению с хранением полного дерева кодирования. Вместо самих кодов сохраняются только их длины.

Построение канонических кодов выполняется после сортировки символов по длине кодов. Первый код минимальной длины принимается равным нулю, а каждый следующий код получается последовательным увеличением предыдущего значения.

Использование канонических кодов упрощает хранение таблицы кодирования и ускоряет процесс декодирования.

2 Практическая часть

Для ускорения проведения параметрических исследований и построения графиков использовались первые 20 000 байт каждого тестового файла. При этом

проверка корректности компрессии и декомпрессии выполнялась на исходных тестовых файлах полностью.

В ходе выполнения лабораторной работы были реализованы и исследованы алгоритмы сжатия данных:

1. Huffman coding (HA);
2. Run-Length Encoding (RLE);
3. BWT + RLE;
4. BWT + MTF + HA;
5. BWT + MTF + RLE + HA;
6. LZSS;
7. LZSS + HA;
8. LZW;
9. LZW + HA.

Для каждого алгоритма были реализованы процедуры компрессии и декомпрессии. Проверка корректности выполнялась сравнением исходных данных и результата после полного цикла:

compress $\rightarrow$ decompress.

Во всех проведенных экспериментах после декомпрессии были получены данные, полностью совпадающие с исходными.

2.1 Тестовые данные

Исследование эффективности алгоритмов проводилось на нескольких типах данных: текстовом файле `enwik7`, текстовом файле `text.txt`, случайных бинарных данных `binary.bin`, а также на RAW-изображениях: черно-белом, grayscale и цветном.

Для изображений предварительно выполнялось преобразование в RAW-формат, после чего к данным применялись реализованные компрессоры.

2.2 Преобразование изображений в RAW-формат

Для изображений был реализован собственный RAW-формат, содержащий метаданные о типе изображения (черно-белое, grayscale или цветное), а также размеры изображения.

После преобразования изображений в RAW-формат было выполнено сравнение размеров исходных файлов и полученных RAW-файлов. Было установлено, что изображения в форматах JPEG и PNG занимают значительно меньший объем памяти благодаря встроенным алгоритмам сжатия. RAW-файлы содержат данные без сжатия, поэтому их размер существенно больше.

2.3 Исследование размера блока BWT

Для алгоритмов, использующих BWT, было исследовано влияние размера блока на энтропию данных после преобразований BWT+MTF.

Были исследованы размеры блоков:

128, 256, 512, 1024, 2048 байт.

Результаты исследования представлены в таблице.

На рисунке 1 представлена зависимость энтропии сообщения после BWT+MTF от размера блока BWT для тестовых файлов. Также выполнялось сравнение с исходной энтропией данных.

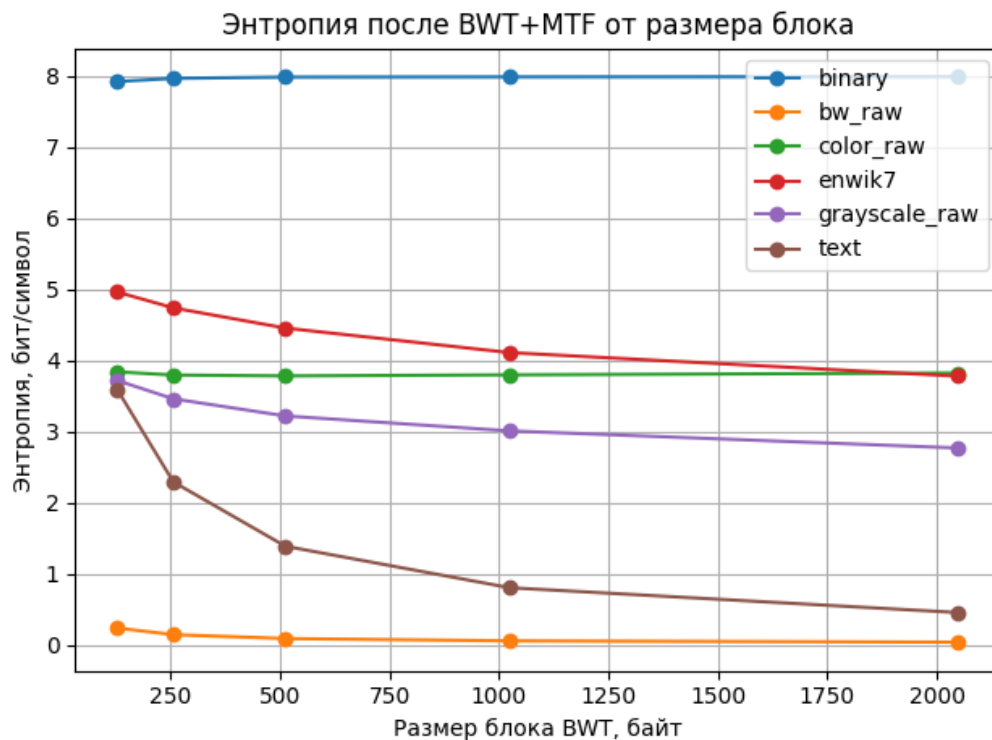


Рисунок 1 – Зависимость энтропии после BWT+MTF от размера блока BWT

Таблица 1: Исследование размера блока BWT

Файл	Лучший размер блока	Минимальная энтропия
enwik7	2048	3.7797
text	2048	0.4558
binary	128	7.9230
bw_raw	2048	0.0390
grayscale_raw	2048	2.7697
color_raw	512	3.7834

Из результатов видно, что увеличение размера блока BWT обычно при-

водит к уменьшению энтропии данных, особенно для текстовых файлов и изображений. Однако увеличение размера блока также увеличивает время выполнения алгоритма.

В качестве компромисса между степенью сжатия и производительностью был выбран размер блока:

2048 байт

2.4 Исследование параметров LZSS

Для алгоритма LZSS было исследовано влияние размера буфера поиска на коэффициент сжатия.

Были протестированы размеры буфера:

32, 64, 128, 256, 512 байт.

Результаты представлены в таблице.

На рисунке 2 представлена зависимость коэффициента сжатия от размера буфера для алгоритма LZSS.

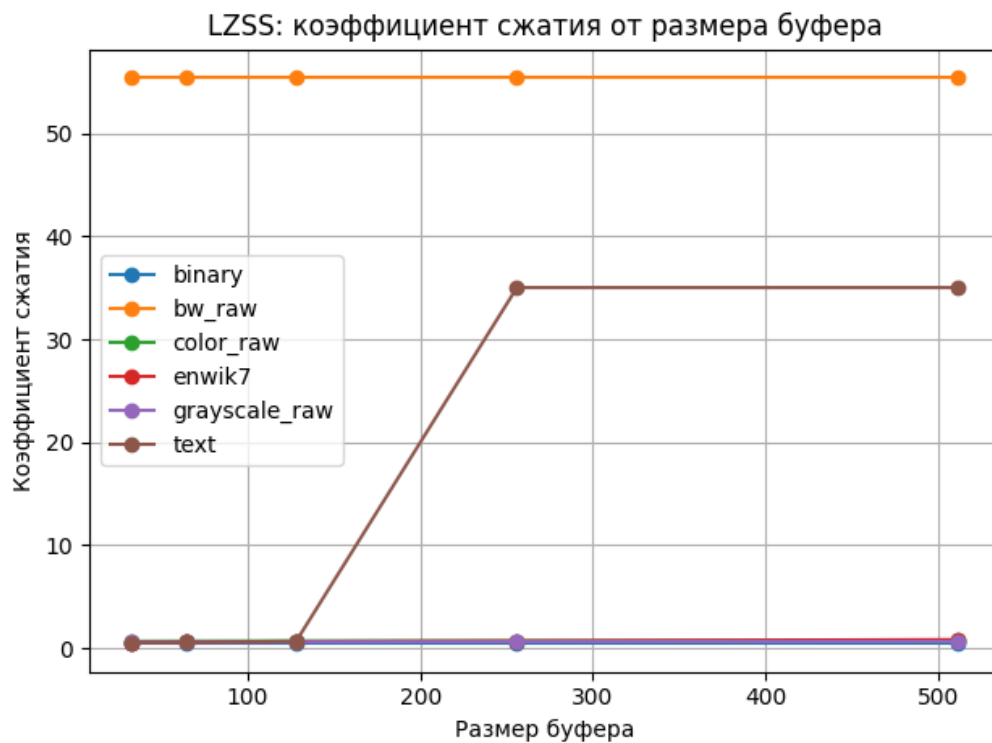


Рисунок 2 – Зависимость коэффициента сжатия LZSS от размера буфера

Таблица 2: Исследование размера буфера LZSS

Файл	Лучший размер буфера	Коэффициент сжатия
enwik7	512	0.8466
text	256	35.0263
binary	32	0.4998
bw_raw	32	55.4017
grayscale_raw	512	0.6704
color_raw	512	0.7416

При увеличении размера буфера алгоритм способен находить более длинные совпадения, благодаря чему коэффициент сжатия увеличивается. Одновременно с этим возрастает время работы алгоритма.

Для дальнейших экспериментов был выбран размер буфера:

512 байт

2.5 Исследование параметров LZW

Для алгоритма LZW было исследовано влияние максимального размера словаря на коэффициент сжатия.

Были исследованы размеры словаря:

512, 1024, 2048, 4096, 8192, 16384, 32768, 65535.

Результаты приведены в таблице.

На рисунке 3 представлена зависимость коэффициента сжатия от размера словаря для алгоритма LZW.

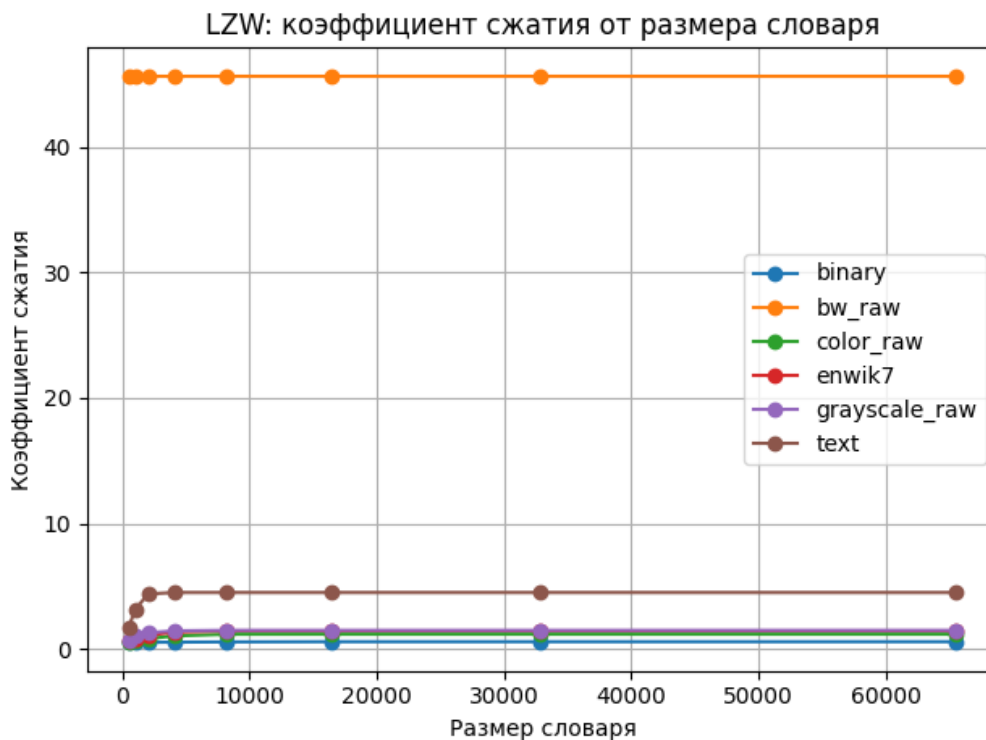


Рисунок 3 – Зависимость коэффициента сжатия LZW от размера словаря

Таблица 3: Исследование размера словаря LZW

Файл	Лучший размер словаря	Коэффициент сжатия
enwik7	8192	1.4294
text	4096	4.4843
binary	32768	0.5627
bw_raw	512	45.6621
grayscale_raw	8192	1.4697
color_raw	16384	1.1575

Увеличение размера словаря обычно улучшает степень сжатия, особенно для текстовых данных и данных с большим количеством повторов. После достижения некоторого размера словаря улучшение коэффициента сжатия становится незначительным.

Для дальнейших экспериментов был выбран размер словаря:

8192

2.6 Проверка корректности компрессии и декомпрессии

Для всех реализованных компрессоров была проведена проверка корректности работы. После декомпрессии были получены данные, полностью совпадающие с исходными.

Результаты работы компрессоров представлены в таблице.

Таблица 4: Результаты работы компрессоров

Файл	Компрессор	Исходный размер	Размер после сжатия	Коэффициент
enwik7	HA	20000	12870	1.5540
enwik7	BWT+MTF+HA	20000	10797	1.8524
enwik7	LZW	20000	13992	1.4294
enwik7	LZSS+HA	20000	11953	1.6732
text	BWT+MTF+RLE+HA	20000	3117	6.4164
text	LZSS+HA	20000	386	51.8135
text	LZW	20000	4460	4.4843
binary	HA	20000	20534	0.9740
binary	LZW	20000	35544	0.5627
binary	LZSS	20000	40011	0.4999
bw_raw	RLE	20000	345	57.9710
bw_raw	BWT+MTF+RLE+HA	20000	289	69.2042
bw_raw	LZSS+HA	20000	152	131.5789
grayscale_raw	BWT+MTF+HA	20000	8048	2.4851
grayscale_raw	LZW+HA	20000	12204	1.6388
color_raw	BWT+MTF+HA	20000	10195	1.9617
color_raw	LZW+HA	20000	15518	1.2888

2.7 Выводы

В ходе выполнения работы были реализованы и исследованы различные алгоритмы сжатия данных.

Эксперименты показали, что алгоритмы семейства BWT хорошо работают на текстовых данных и изображениях. Алгоритм RLE особенно эффективен для данных с длинными последовательностями одинаковых символов. Алгоритмы LZSS и LZW показывают хорошие результаты на структурированных данных с повторяющимися фрагментами.

Также было установлено, что комбинация нескольких методов сжатия обычно позволяет получить более высокий коэффициент сжатия по сравне-

нию с использованием одного алгоритма.

Наилучшие результаты были получены для:

LZSS+HA

на текстовых данных и для

BWT+MTF+RLE+HA

на изображениях.

Случайные бинарные данные практически не поддаются сжатию, что соответствует теоретическим ожиданиям.

Все реализованные компрессоры успешно прошли проверку корректности компрессии и декомпрессии.

3 Приложение

Github

Ссылка: https://github.com/diwkos/Algosi/blob/main/1lab_4sem

Код программы

```
from pathlib import Path
import time
```

```
# Основной файл с алгоритмами.
```

```
# Если он у тебя называется иначе, поменяй импорт.
```

```
import lab1_compression as lab
```

```
DATA_DIR = Path("data")
```

```

RESULTS_DIR = Path("results")
RESULTS_DIR.mkdir(exist_ok=True)

# Файлы для проверки.
# Если названия другие - поменяй тут.
TEST_FILES = {
    "enwik7": DATA_DIR / "enwik7",
    "text": DATA_DIR / "text.txt",
    "binary": DATA_DIR / "binary.bin",
    "bw_raw": DATA_DIR / "bw.raw",
    "grayscale_raw": DATA_DIR / "grayscale.raw",
    "color_raw": DATA_DIR / "color.raw",
}

# Картинки, из которых создаются raw.
# Если у тебя другие расширения, поменяй .jpeg/.png/.jpg.
IMAGES = [
    (DATA_DIR / "bw.jpeg", DATA_DIR / "bw.raw", lab.RAW_BW),
    (DATA_DIR / "grayscale.png", DATA_DIR / "grayscale.raw",
     ↪ lab.RAW_GRAY),
    (DATA_DIR / "color.png", DATA_DIR / "color.raw", lab.RAW_RGB),
]

# Чтобы программа не работала часами, для графиков берем начало
↪ файла.
# Для отчета это нормально: исследование зависимости параметров.
DATA_LIMIT = 20_000

```

```
BWT_BLOCKS = [128, 256, 512, 1024, 2048]
LZSS_WINDOWS = [32, 64, 128, 256, 512]
LZW_DICTS = [512, 1024, 2048, 4096, 8192, 16384, 32768, 65535]
```

```
def read_data(path):
    data = path.read_bytes()
    if len(data) > DATA_LIMIT:
        data = data[:DATA_LIMIT]
    return data
```

```
def prepare_raw():
    print("Создание raw-файлов")
    for inp, out, img_type in IMAGES:
        if inp.exists():
            lab.image_to_raw(inp, out, img_type)
            print("OK:", out)
        else:
            print("Нет файла:", inp)
```

```
def plot_lines(all_rows, x_key, y_key, filename, title, xlabel,
    ↪ ylabel):
    import matplotlib.pyplot as plt
```

```

plt.figure()

files = sorted(set(row["file"] for row in all_rows))
for file_name in files:
    rows = [row for row in all_rows if row["file"] ==
            ↪ file_name]
    xs = [row[x_key] for row in rows]
    ys = [row[y_key] for row in rows]
    plt.plot(xs, ys, marker="o", label=file_name)

plt.title(title)
plt.xlabel(xlabel)
plt.ylabel(ylabel)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(RESULTS_DIR / filename)
plt.close()

print("График сохранен:", RESULTS_DIR / filename)

def experiment_entropy():
    print("\n1. Энтропия после BWT+MTF")

    rows = []
    conclusions = []

```

```

for name, path in TEST_FILES.items():
    if not path.exists():
        print("Файл не найден:", path)
        continue

    data = read_data(path)
    original_entropy = lab.calculate_entropy(data, 1)

    best_block = None
    best_entropy = None

    for block in BWT_BLOCKS:
        transformed =
            ↪ lab.mtf_encode(lab.bwt_encode_blocks(data, block))
        ent = lab.calculate_entropy(transformed, 1)

        rows.append({
            "file": name,
            "block_size": block,
            "entropy": ent,
            "original_entropy": original_entropy,
        })

    if best_entropy is None or ent < best_entropy:
        best_entropy = ent
        best_block = block

```

```

        conclusions.append(
            f"{name}: исходная энтропия {original_entropy:.4f}, "
            f"минимальная после BWT+MTF {best_entropy:.4f}, "
            f"лучший размер блока {best_block} байт."
        )

    plot_lines(
        rows,
        "block_size",
        "entropy",
        "graph_1_entropy_bwt_mtf.png",
        "Энтропия после BWT+MTF от размера блока",
        "Размер блока BWT, байт",
        "Энтропия, бит/символ",
    )

    return conclusions


def experiment_lzss():
    print("\n2. LZSS: коэффициент сжатия от размера буфера")

    rows = []
    conclusions = []

    for name, path in TEST_FILES.items():

```

```

if not path.exists():
    print("Файл не найден:", path)
    continue

data = read_data(path)

best_window = None
best_ratio = None

for window in LZSS_WINDOWS:
    encoded = lab.compressor_lzss_encode(data,
        ↪ window_size=window)
    decoded = lab.compressor_lzss_decode(encoded)

    if decoded != data:
        raise RuntimeError("Ошибка декодирования LZSS")

    ratio = lab.compression_ratio(len(data), len(encoded))

    rows.append({
        "file": name,
        "window_size": window,
        "ratio": ratio,
    })

    if best_ratio is None or ratio > best_ratio:
        best_ratio = ratio

```

```

        best_window = window

    conclusions.append(
        f"{name}: лучший размер буфера LZSS {best_window}
        ↪ байт, "
        f"коэффициент сжатия {best_ratio:.4f}."
    )

plot_lines(
    rows,
    "window_size",
    "ratio",
    "graph_2_lzss_window.png",
    "LZSS: коэффициент сжатия от размера буфера",
    "Размер буфера",
    "Коэффициент сжатия",
)

return conclusions

def experiment_lzw():
    print("\n3. LZW: коэффициент сжатия от размера словаря")

    rows = []
    conclusions = []

```



```

for name, path in TEST_FILES.items():
    if not path.exists():
        print("Файл не найден:", path)
        continue

    data = read_data(path)

    best_dict = None
    best_ratio = None

    for dict_size in LZW_DICTS:
        encoded = lab.compressor_lzw_encode(data,
            ↪ max_dict_size=dict_size)
        decoded = lab.compressor_lzw_decode(encoded)

        if decoded != data:
            raise RuntimeError("Ошибка декодирования LZW")

        ratio = lab.compression_ratio(len(data), len(encoded))

        rows.append({
            "file": name,
            "dict_size": dict_size,
            "ratio": ratio,
        })

    if best_ratio is None or ratio > best_ratio:

```

```

        best_ratio = ratio
        best_dict = dict_size

    conclusions.append(
        f"{name}: лучший размер словаря LZW {best_dict}, "
        f"коэффициент сжатия {best_ratio:.4f}."
    )

plot_lines(
    rows,
    "dict_size",
    "ratio",
    "graph_3_lzw_dictionary.png",
    "LZW: коэффициент сжатия от размера словаря",
    "Размер словаря",
    "Коэффициент сжатия",
)

return conclusions

def check_all_compressors_txt():
    """
    Эта часть делает только txt-файл:
    проверяет 9 компрессоров и записывает размеры.
    Без CSV.
    """

```

```

print("\nПроверка 9 компрессоров, результат будет в txt")

lines = []
lines.append("Проверка компрессоров\n")
lines.append("Файл | Компрессор | Исходный размер | Размер
↪ после сжатия | Размер после распаковки | Коэффициент |
↪ Проверка\n")
lines.append("-" * 120 + "\n")

for file_name, path in TEST_FILES.items():
    if not path.exists():
        continue

    data = read_data(path)

    for comp_name, pair in lab.COMPRESSORS.items():
        enc, dec = pair

        try:
            compressed = enc(data)
            restored = dec(compressed)
            ok = restored == data

            line = (
                f"{file_name} | {comp_name} | "
                f"{len(data)} | {len(compressed)} | "
                ↪ f"{len(restored)} | "

```

```

        f"{lab.compression_ratio(len(data),
        ↪ len(compressed)):.4f} | "
        f"'OK' if ok else 'ERROR'\n"
    )
    lines.append(line)

except Exception as e:
    lines.append(f"{file_name} | {comp_name} | ERROR:
    ↪ {e}\n")

path = RESULTS_DIR / "compressors_result_table.txt"
path.write_text("".join(lines), encoding="utf-8")
print("TXT сохранен:", path)

def main():
    print("Запуск короткой версии экспериментов")

    lab.run_tests()

    prepare_raw()

    conclusions = []
    conclusions.append("Выводы по графикам\n")
    conclusions.append("=" * 60 + "\n\n")

    conclusions.append("1. Энтропия BWT+MTF\n")

```

```

for line in experiment_entropy():
    conclusions.append(line + "\n")
conclusions.append(
    "\nВывод: при увеличении размера блока BWT энтропия после
    ↪ BWT+MTF обычно уменьшается, "
    "но слишком большой блок заметно увеличивает время работы.
    ↪ Поэтому оптимальным можно считать "
    "тот размер блока, при котором энтропия уже близка к
    ↪ минимальной, а программа еще работает нормально.\n\n"
)

```

```

conclusions.append("2. LZSS\n")
for line in experiment_lzss():
    conclusions.append(line + "\n")
conclusions.append(
    "\nВывод: при увеличении буфера LZSS алгоритм чаще находит
    ↪ длинные совпадения, "
    "поэтому коэффициент сжатия обычно растет. Но большой
    ↪ буфер увеличивает время работы.\n\n"
)

```

```

conclusions.append("3. LZW\n")
for line in experiment_lzw():
    conclusions.append(line + "\n")
conclusions.append(
    "\nВывод: больший словарь LZW обычно улучшает сжатие,
    ↪ особенно для текста и данных с повторами. "
)

```

```

        "После некоторого размера словаря улучшение становится
        ↪ небольшим.\n\n"
    )

    (RESULTS_DIR /
     ↪ "graphs_conclusions.txt").write_text("".join(conclusions),
     ↪ encoding="utf-8")
    print("Выводы сохранены:", RESULTS_DIR /
     ↪ "graphs_conclusions.txt")

    check_all_compressors_txt()

    print("\nГотово.")
    print("Тебе нужны эти файлы:")
    print("1) results/graph_1_entropy_bwt_mtf.png")
    print("2) results/graph_2_lzss_window.png")
    print("3) results/graph_3_lzw_dictionary.png")
    print("4) results/graphs_conclusions.txt")
    print("5) results/compressors_result_table.txt")

if __name__ == "__main__":
    main()

```